

Interview Questions Frontend

Generated on February 05, 2026 at 01:29 PM | 60 entries

Q1. Explain the difference between block-level elements and inline-level elements in HTML, providing an example of each.

[EASY] | Category: Frontend Developer

Answer: **Block-level elements** (like `<div>`, `<p>`, `<h1>`) always start on a new line and take up the full width available to them. They can contain other block or inline elements. **Inline-level elements** (like `<span>`, `<a>`, `<img>`) do not start on a new line and only take up as much width as their content requires. They typically exist *within* block-level elements.

Q2. What are the main differences between the `<ol>` (ordered list) and `<ul>` (unordered list) HTML elements?

[EASY] | Category: Frontend Developer

Answer: Both elements are used to group a list of items using `<li>` tags. The key difference is presentation and meaning. An `<ol>` (ordered list) is used when the *order* of the items is important, and it displays them with numbers by default (e.g., step-by-step instructions). An `<ul>` (unordered list) is used when the order does not matter, and it displays them with bullet points by default (e.g., a list of features).

Q3. What is the purpose of semantic HTML, and why is it more important than just using `<div>` tags for everything?

[EASY] | Category: Frontend Developer

Answer: Semantic HTML uses tags that describe the *meaning* and *purpose* of the content they contain, such as `<header>`, `<nav>`, `<article>`, and `<footer>`. This is crucial for two reasons: **Accessibility (a11y)**, as it allows screen readers to understand the page structure, and **Search Engine Optimization (SEO)**, as it helps search engines index the content more effectively. Using only `<div>` tags provides no context about the content.

Q4. Can you describe the difference between the <html>, <head>, and <body> tags in an HTML document?

[EASY] | Category: Frontend Developer

Answer: The `<html>` tag is the root element that wraps all other content on the page. The `<head>` tag contains meta-information *about* the document, such as the page title (`<title>`), links to stylesheets (`<link>`), and character encoding. This information is not visibly displayed. The `<body>` tag contains all the visible content of the web page, such as text, images, and links.

Q5. What is the 'alt' attribute on an tag used for, and why is it crucial for accessibility?

[EASY] | Category: Frontend Developer

Answer: The `alt` (alternative text) attribute provides a textual description of an image. It is crucial for accessibility because it's what screen readers announce to visually impaired users, allowing them to understand the image's content. It also serves as a placeholder if the image fails to load and helps with SEO by providing context to search engines.

Q6. Can you explain the four components of the CSS box model and how they interact with each other?

[EASY] | Category: Frontend Developer

Answer: The CSS box model describes the rectangular boxes that are generated for elements. The four components, from the inside out, are: **1. Content:** The text, image, or other content. **2. Padding:** The transparent space *around* the content, inside the border. **3. Border:** The line that goes *around* the padding and content. **4. Margin:** The transparent space *outside* the border, which separates the element from other elements.

Q7. What is CSS specificity, and how does the browser decide which CSS rule to apply when there are conflicts?

[EASY] | Category: Frontend Developer

Answer: CSS Specificity is the set of rules a browser uses to determine which CSS style declaration is the most 'specific' and therefore should be applied to an element. It's a weighted system. From most specific to least: **Inline Styles** (highest), **IDs**, **Classes/Attributes/Pseudo-classes**, and finally **Elements/Pseudo-elements** (lowest). A more specific selector will always override a less specific one.

Q8. What is the difference between 'padding' and 'margin' in CSS, and when would you use one over the other?

[EASY] | Category: Frontend Developer

Answer: Both create space, but their location differs based on the CSS box model. **Padding** is the space *inside* an element's border, between the border and the content. You use it to give the content 'room to breathe' within its box. **Margin** is the space *outside* an element's border. You use it to create space *between* that element and other elements on the page.

Q9. Explain the three main ways to add CSS to an HTML page (inline, internal, and external stylesheets).

[EASY] | Category: Frontend Developer

Answer: **1. Inline:** The CSS is written directly inside an HTML tag using the `style` attribute (e.g., `<p style="color: red;">`). This is bad practice. **2. Internal:** The CSS is written inside a `<style>` tag within the HTML's `<head>` section. **3. External:** The CSS is written in a separate `.css` file and linked from the HTML's `<head>` section using a `<link>` tag. This is the best practice for maintainability and caching.

Q10. Describe the difference between 'class' selectors and 'id' selectors in CSS. When should you use each?

[EASY] | Category: Frontend Developer

Answer: Both are used to select HTML elements. An **ID selector** (e.g., `#header`) is used to select *one unique element* on the page. An ID must be unique. A **Class selector** (e.g., `.button`) is used to select *one or more elements* that share a common style. You can apply the same class to many elements. You should use classes for reusable styles and IDs for unique page landmarks.

Q11. What is the 'z-index' property in CSS, and how does it affect the stacking order of elements?

[EASY] | *Category: Frontend Developer*

Answer: The `z-index` property controls the vertical stacking order (which element is 'in front' or 'behind' another) of elements that have a 'position' value other than 'static' (e.g., 'relative', 'absolute', or 'fixed'). An element with a higher `z-index` value will be displayed in front of an element with a lower `z-index` value.

Q12. What is the difference between 'visibility: hidden' and 'display: none' in CSS?

[EASY] | *Category: Frontend Developer*

Answer: Both properties make an element invisible. `display: none` completely removes the element from the document flow. It takes up no space, and other elements will 'reflow' to fill its spot. `visibility: hidden` simply makes the element invisible, but it *still occupies its original space* in the layout. The page layout does not change.

Q13. What is the difference between 'let', 'const', and 'var' when declaring variables in JavaScript?

[EASY] | Category: Frontend Developer

Answer: `var` is the old, function-scoped way to declare variables. It's generally avoided due to its 'hoisting' behavior. `let` and `const` were introduced in ES6 and are 'block-scoped' (they only exist within the curly braces they are defined in). `let` allows you to declare a variable that can be reassigned later. `const` declares a variable that *cannot* be reassigned, making it ideal for constant values.

Q14. Explain the difference between '==' (loose equality) and '===' (strict equality) operators in JavaScript.

[EASY] | Category: Frontend Developer

Answer: **Strict equality (`===`)** is generally preferred. It checks if two values are equal in *both type and value* (e.g., `7 === '7'` is false). **Loose equality (`==`)** will first try to 'coerce' (convert) the values to a common type before comparing them (e.g., `7 == '7'` is true). This type coercion can lead to unpredictable bugs and is best avoided.

Q15. What is the DOM (Document Object Model), and how does JavaScript interact with it to change a webpage?

[EASY] | Category: Frontend Developer

Answer: The DOM is a tree-like representation of the HTML document that the browser creates in memory. It allows JavaScript to interact with the page's content and structure. JavaScript can use the DOM to: **find** elements (e.g., `document.getElementById()`), **change** elements (e.g., `element.innerHTML = 'New text'`), **add** new elements, and **remove** elements, making the webpage dynamic and interactive.

Q16. What are the primitive data types in JavaScript? Name at least five of them.

[EASY] | Category: Frontend Developer

Answer: A primitive data type is data that is not an object and has no methods. JavaScript has seven primitive types: **String** (for text), **Number** (for all numbers, including integers and decimals), **Boolean** (for 'true' or 'false' values), **Undefined** (a variable that has been declared but not assigned a value), **Null** (an intentional 'nothing' value), **BigInt** (for very large integers), and **Symbol** (for unique identifiers).

Q17. What is a JavaScript 'function', and what is the difference between a function declaration and a function expression?

[EASY] | *Category: Frontend Developer*

Answer: A function is a reusable block of code that performs a specific task. A **function declaration** is defined using the `function` keyword followed by a name (e.g., `function foo() {}`). These are 'hoisted', meaning they are available throughout their scope. A **function expression** involves creating a function and assigning it to a variable (e.g., `const foo = function() {}`). These are **not** hoisted and can only be used after they are defined.

Q18. How does an 'event listener' work in JavaScript? Provide an example like a 'click' event.

[EASY] | *Category: Frontend Developer*

Answer: An event listener is a function that 'listens' for a specific event (like a 'click', 'mouseover', or 'keydown') to occur on a specific HTML element. When that event happens, the function is executed. For example, you can select a button with `document.getElementById('myButton')` and then attach a listener with `button.addEventListener('click', function() { alert('Button was clicked!'); })`.

Q19. What is the difference between 'null' and 'undefined' in JavaScript?

[EASY] | *Category: Frontend Developer*

Answer: Both `null` and `undefined` represent the absence of a value, but they are used in different contexts. `undefined` is the default value for a variable that has been declared but not yet assigned a value, or for a function that doesn't return anything. `null` is an *intentional* value. A developer explicitly assigns `null` to a variable to signify that it holds 'no value' or 'nothing'.

Q20. What is JSON (JavaScript Object Notation), and how is it used for fetching data in frontend applications?

[EASY] | *Category: Frontend Developer*

Answer: JSON is a lightweight, text-based data interchange format that is easy for humans to read and for machines to parse. It's the standard format for transferring data between a server and a frontend application (e.g., in an API response). A frontend application will use the 'Fetch API' to request data, and the server will send back a JSON string, which JavaScript can easily parse into a native JavaScript object using `JSON.parse()`.

Q21. What are CSS media queries, and how are they used to create responsive (mobile-first) website designs?

[MEDIUM] | *Category: Frontend Developer*

Answer: Media queries are a CSS feature that allows you to apply different styles based on the device's characteristics, most commonly the viewport width. This is the foundation of responsive design. In a 'mobile-first' approach, you write the default CSS for a small mobile screen, and then use media queries (e.g., `@media (min-width: 768px) { ... }`) to add or override styles for larger screens like tablets and desktops.

Q22. Explain the core differences between CSS Flexbox and CSS Grid. When would you choose one over the other?

[MEDIUM] | *Category: Frontend Developer*

Answer: Both are modern CSS layout models. **Flexbox** is designed for **one-dimensional** layouts—either in a row *or* a column. It's perfect for aligning items within a container, like a navigation bar or a row of cards. **CSS Grid** is designed for **two-dimensional** layouts—both rows *and* columns simultaneously. It's ideal for complex, page-wide layouts, allowing you to define a 'grid' and place items precisely within it.

Q23. What is a CSS preprocessor like 'Sass' or 'LESS', and what key benefits do they provide over writing plain CSS?

[MEDIUM] | *Category: Frontend Developer*

Answer: A CSS preprocessor is a scripting language that extends CSS and is compiled into regular CSS. They provide powerful features that plain CSS lacks, making CSS more maintainable. Key benefits include: **Variables** (to store reusable values like colors), **Nesting** (to structure rules similar to HTML), **Mixins** (for reusable blocks of styles), and **Functions** (to perform calculations).

Q24. What is the difference between 'position: absolute', 'position: relative', and 'position: fixed' in CSS?

[MEDIUM] | *Category: Frontend Developer*

Answer: **position: relative** offsets an element from its normal position in the document flow, but it still occupies its original space. **position: absolute** removes the element from the document flow and positions it *relative to its nearest positioned ancestor* (an ancestor with a position other than 'static'). **position: fixed** removes the element from the document flow and positions it *relative to the browser viewport*. This means it will stay in the same place even when the page is scrolled.

Q25. What is the BEM (Block, Element, Modifier) methodology, and how does it help in writing scalable and maintainable CSS?

[MEDIUM] | *Category: Frontend Developer*

Answer: BEM is a naming convention for CSS classes that helps create independent, reusable components. ****Block:**** The standalone component (e.g., `.card``). **\*\*Element:\*\*** A part of the block that has no meaning on its own (e.g., `.card__title``). ****Modifier:**** A flag on a block or element that changes its appearance or behavior (e.g., `.card--large`` or `.card__title--primary``). This naming convention prevents CSS conflicts, makes code self-documenting, and is highly scalable.

Q26. How does the 'box-sizing: border-box' CSS property change the behavior of the standard box model?

[MEDIUM] | *Category: Frontend Developer*

Answer: By default (`box-sizing: content-box``), an element's specified `width`` and `height`` apply only to its *content*. The `padding`` and `border`` are *added on top* of that, making the element visually larger than its set width. **`box-sizing: border-box``** changes this. It makes the `width`` and `height`` properties apply to the *entire* element, *including* its padding and border. This is a much more intuitive way to size elements.

Q27. What is a 'closure' in JavaScript, and can you provide a practical example of when one might be used?

[MEDIUM] | *Category: Frontend Developer*

Answer: A closure is a function that 'remembers' and has access to variables from its outer (enclosing) scope, even *after* that outer function has finished executing. A practical example is a 'counter' function: you can have an outer function that defines a `count`` variable, and returns an inner function. This inner function increments and returns the `count``. The `count`` variable is 'closed over' and persists, protected from the global scope.

Q28. Explain the concept of 'hoisting' in JavaScript. How does it affect variable and function declarations?

[MEDIUM] | *Category: Frontend Developer*

Answer: Hoisting is JavaScript's default behavior of moving all declarations to the top of their current scope *before* code execution. This means `var`` variable declarations are 'hoisted' (but their assignments are not), which is why they are `undefined`` until assigned. ****Function declarations**** are fully hoisted, meaning you can call a function *before* it is defined in the code. `let`` and `const`` variables are *not* hoisted, creating a 'temporal dead zone' and throwing an error if accessed before declaration.

Q29. What are JavaScript 'Promises', and how do they help manage asynchronous operations better than traditional callbacks?

[MEDIUM] | *Category: Frontend Developer*

Answer: A Promise is an object that represents the eventual completion (or failure) of an asynchronous operation. Instead of 'callback hell' (deeply nested callbacks), Promises allow you to chain operations using `.then()` for success and `.catch()` for errors. This makes asynchronous code much cleaner, more readable, and easier to reason about, especially when handling multiple sequential or parallel async tasks.

Q30. What is the difference between 'localStorage', 'sessionStorage', and 'cookies' for storing data in a browser?

[MEDIUM] | *Category: Frontend Developer*

Answer: **Cookies** are small strings of data sent with *every* HTTP request, primarily used for server-side session management. **localStorage** and **sessionStorage** are part of the 'Web Storage API' and are *not* sent to the server. **localStorage** persists data permanently until it's explicitly cleared. **sessionStorage** only persists data for the duration of the page *session* (it's cleared when the tab is closed).

Q31. Can you explain what 'async/await' keywords do in JavaScript and how they improve the readability of asynchronous code?

[MEDIUM] | Category: Frontend Developer

Answer: `async/await` is 'syntactic sugar' built on top of Promises, making asynchronous code *look and feel* synchronous. An `async` function always returns a Promise. The `await` keyword can be used *inside* an `async` function to 'pause' its execution and wait for a Promise to resolve before moving to the next line. This completely eliminates the need for `.then()` chains and makes complex asynchronous logic as easy to read as a simple, top-down script.

Q32. What are ES6 'arrow functions', and how does their handling of the 'this' keyword differ from regular functions?

[MEDIUM] | Category: Frontend Developer

Answer: Arrow functions (`=>`) provide a more concise syntax for writing functions. Their main behavioral difference is how they handle the `this` keyword. A regular `function` gets its *own* `this` context, which is set by *how the function is called*. An *arrow function* does *not* have its own `this`; it *lexically inherits* `this` from its *parent* or *enclosing* scope. This is extremely useful in contexts like class methods or array iterators, as it avoids common `this`-binding issues.

Q33. What is 'event delegation' (or 'event bubbling') in JavaScript, and why is it a useful pattern?

[MEDIUM] | Category: Frontend Developer

Answer: Event bubbling is the DOM's default behavior where an event (like a 'click') on a child element 'bubbles up' to its parent, then its grandparent, and so on, up to the `document`. *Event delegation* is a pattern that leverages this. Instead of adding many event listeners to *individual children* (e.t., 100 list items), you add *one* listener to the *parent* container. This single listener can then check the `event.target` property to see which child was actually clicked. This is far more performant and scales better.

Q34. Explain the difference between `Array.map()`, `Array.filter()`, and `Array.reduce()` methods in JavaScript.

[MEDIUM] | Category: Frontend Developer

Answer: These are all array iteration methods. `.map()` creates a *new array* by transforming every element in the original array (e.t., multiply all numbers by 2). `.filter()` creates a *new array* containing only the elements that pass a test (e.g., keep only the numbers greater than 10). `.reduce()` 'reduces' the entire array down to a *single value* by applying a function to an accumulator and each element (e.g., finding the *sum* of all numbers in the array).

Q35. What is the 'Fetch API', and how would you use it to make a simple 'GET' request to a server?

[MEDIUM] | *Category: Frontend Developer*

Answer: The Fetch API is a modern, Promise-based interface for making HTTP requests in the browser. It's a more powerful and flexible replacement for `XMLHttpRequest`. To make a simple 'GET' request, you call `fetch('https://api.example.com/data')`, which returns a Promise. You then use `.then()` to handle the response, and a second `.then()` to parse the response body as JSON (e.g., `response.json()`), and a `.catch()` block to handle any network errors.

Q36. What is the 'this' keyword in JavaScript, and how is its value determined in different contexts?

[MEDIUM] | *Category: Frontend Developer*

Answer: The `this` keyword is a reference to the 'execution context' of a function. Its value is not static; it's determined by *how the function is called*.
1. Global Context: `this` refers to the `window` object.
2. As an Object Method: `this` refers to the *object* the method was called on (e.g., `user.sayHi()`, `this` is `user`).
3. As a Constructor: `this` refers to the *new instance* being created.
4. Arrow Functions: `this` is lexically inherited from the parent scope.

Q37. What is a 'component' in a modern frontend framework like React, and what is the difference between 'state' and 'props'?

[MEDIUM] | *Category: Frontend Developer*

Answer: A 'component' is a self-contained, reusable piece of the user interface (UI) that encapsulates its own logic and markup (e.g., a `<Button>` or `<UserProfile>` component). **Props (Properties):** Are data passed *down* from a parent component to a child component. They are 'read-only' and cannot be changed by the child. **State:** Is internal data managed *inside* the component itself. It can be changed, and when 'state' changes, the framework (like React) will automatically re-render the component to reflect the new data.

Q38. What is the 'virtual DOM', and how does it help frameworks like React improve rendering performance?

[MEDIUM] | *Category: Frontend Developer*

Answer: The Virtual DOM (VDOM) is a lightweight, in-memory representation of the 'real' browser DOM. When a component's state changes, React first creates a *new* VDOM tree. It then uses a 'diffing' algorithm to compare this new VDOM with the old VDOM, identifying the *minimal* set of changes needed. Finally, it 'patches' the *real* DOM with only those changes. This is much faster than re-rendering the entire real DOM, as DOM manipulations are very slow.

Q39. What is 'npm' (or 'yarn'), and what is its role in the modern frontend development workflow?

[MEDIUM] | *Category: Frontend Developer*

Answer: `npm` (Node Package Manager) is a 'package manager' for JavaScript. Its role is twofold: **1. Library Management:** It allows developers to easily download, install, and manage third-party 'packages' (libraries and tools) that their project depends on. **2. Script Runner:** It's also used as a task runner, defined in the `package.json` file's 'scripts' section, to run common tasks like starting a development server (`npm start`), building the project (`npm run build`), or running tests (`npm test`).

Q40. What is 'prop drilling' in React, and what are some common techniques (like Context API or Redux) to avoid it?

[MEDIUM] | *Category: Frontend Developer*

Answer: 'Prop drilling' is the problem in React where 'state' data needs to be passed down through many layers of nested components, even if the components in the middle don't need the data themselves. This makes the code hard to maintain. To avoid this, you can use: **1. Context API:** A built-in React feature that allows you to 'provide' a value at a high level and 'consume' it in any nested child, without passing props. **2. State Management Libraries (like Redux or Zustand):** These store the application's state in a global 'store' that any component can subscribe to.

Q41. Explain the JavaScript 'event loop' in detail, including the roles of the call stack, microtask queue, and macrotask queue.

[HARD] | *Category: Frontend Developer*

Answer: The Event Loop is JavaScript's concurrency model. The **Call Stack** is a last-in, first-out stack for function execution. When an async operation (like `setTimeout` or `fetch`) is called, it's handed off to the Browser API. When it completes, its callback is placed in one of two queues: **Macrotask Queue** (for `setTimeout`, `setInterval`) or **Microtask Queue** (for `Promises`, `MutationObserver`). The Event Loop's job is to wait until the Call Stack is *empty*, then it processes *all* tasks in the Microtask Queue, and *then* it processes *one* task from the Macrotask Queue. This cycle repeats.

Q42. What is 'prototypal inheritance' in JavaScript, and how does it differ from 'classical inheritance' found in languages like Java or C++?

[HARD] | *Category: Frontend Developer*

Answer: JavaScript uses prototypal inheritance. Every object has a hidden `[[Prototype]]` property that links to another object (its 'prototype'). When you try to access a property on an object, JavaScript first checks the object itself, and if it can't find it, it searches up the 'prototype chain'. **Classical inheritance** (used in Java/C++) is about creating 'blueprints' (classes) that are then instantiated. Prototypal inheritance is about objects 'inheriting' directly from other objects.

Q43. What is a 'debounce' function, and what is a 'throttle' function? Provide a real-world example for each.

[HARD] | *Category: Frontend Developer*

Answer: Both are techniques to control how often a function is executed. **Debounce:** Bundles a series of rapid-fire events (like 'keyup') into *one single* call, which executes only *after* the events have *stopped* for a specified time. **Example:** A search bar, where you wait until the user stops typing to send the API request. **Throttle:** Ensures a function is executed at most *once* per specified time interval, no matter how many times the event fires. **Example:** A 'scroll' event listener, where you only want to run a calculation every 200ms, not on every single pixel scrolled.

Q44. How does garbage collection work in JavaScript, and what is a 'memory leak'? How might you create one?

[HARD] | *Category: Frontend Developer*

Answer: JavaScript uses automatic garbage collection, primarily with a 'Mark-and-Sweep' algorithm. The collector starts from a 'root' object (like the `window`) and 'marks' all objects it can reach. It then 'sweeps' away all 'unmarked' (unreachable) objects, freeing their memory. A **memory leak** occurs when an object is no longer needed but is *still reachable* from the root, so the garbage collector can't remove it. A common way to create one is by adding an event listener to an element (`element.addEventListener(...)`) and then *removing that element from the DOM* without first calling `removeEventListener()`.

Q45. What are 'Web Workers', and how do they allow you to run JavaScript in the background without blocking the main UI thread?

[HARD] | *Category: Frontend Developer*

Answer: JavaScript is single-threaded, meaning long-running tasks (like complex calculations or data processing) will 'block' the main thread, freezing the UI. **Web Workers** solve this by allowing you to run a JavaScript script on a separate background thread. The main thread and the worker thread do *not* share memory; they communicate by passing messages back and forth using the `postMessage()` API. This is ideal for CPU-intensive tasks that would otherwise make the page unresponsive.

Q46. What is 'CORS' (Cross-Origin Resource Sharing), and what are the specific HTTP headers involved in making a cross-origin request work?

[HARD] | *Category: Frontend Developer*

Answer: CORS is a security mechanism enforced by browsers that prevents a frontend on one 'origin' (e.g., `a.com`) from making API requests to a *different* 'origin' (e.g., `b.com`) unless that server explicitly allows it. For a 'preflighted' request (like a `PUT` or `DELETE`), the browser first sends an `OPTIONS` request. The server must respond with the correct headers: **`Access-Control-Allow-Origin`** (which origins are allowed, e.g., `a.com`), **`Access-Control-Allow-Methods`** (e.g., 'PUT, POST'), and **`Access-Control-Allow-Headers`**.

Q47. What is 'strict mode' in JavaScript ('use strict'), and what are the key changes it introduces to the language?

[HARD] | *Category: Frontend Developer*

Answer: 'Strict mode' is an opt-in, restricted variant of JavaScript that makes the language more secure and robust. It's enabled by adding `'use strict';` at the top of a file or function. **Key Changes:** 1. It turns 'silent errors' into *thrown errors* (e.g., assigning a value to an undeclared variable). 2. It *prevents* the use of `this` in the global context (it will be `undefined` instead of `window`). 3. It disallows duplicate property names in objects and duplicate parameter names in functions.

Q48. What is 'code-splitting', and how do tools like Webpack or Vite implement it to improve initial page load performance?

[HARD] | *Category: Frontend Developer*

Answer: Code-splitting is the process of splitting an application's large JavaScript 'bundle' into multiple, smaller 'chunks'. These chunks are then loaded on demand ('lazily') when the user needs them. For example, the code for a 'UserProfile' page is not downloaded until the user clicks the 'Profile' link. This dramatically improves *initial page load time* because the user only has to download the small, critical 'main' chunk, rather than the entire application. Tools like Webpack and Vite can do this automatically using dynamic `import()` statements.

Q49. Explain the 'Context API' in React. How does it work internally, and what are the performance implications of using it?

[HARD] | *Category: Frontend Developer*

Answer: React's Context API provides a way to pass data through the component tree without prop drilling. You create a `Context` object. You use a `Context.Provider` at the top of a tree to pass a `value`, and any nested component can use a `Context.Consumer` or the `useContext` hook to read that value. ****Internally:**** When the `value` in the `Provider` changes, *all* components that consume that context will re-render. This can cause performance issues if the value changes frequently and the consumers are not optimized.

Q50. What are React 'Hooks' (like `useState` and `useEffect`), and how do they change the way you write and manage state in components?

[HARD] | *Category: Frontend Developer*

Answer: Hooks are functions that let you 'hook into' React's state and lifecycle features from *function components*. Before Hooks, you had to use *class components* to manage state or use lifecycle methods. ****`useState`**** allows a function component to declare and manage its own state. ****`useEffect`**** allows you to perform 'side effects' (like data fetching or subscribing to events) after the component renders, replacing the need for lifecycle methods like `componentDidMount` and `componentDidUpdate`.

Q51. What is 'hydration' in the context of Server-Side Rendering (SSR) frameworks like Next.js, and why is it necessary?

[HARD] | *Category: Frontend Developer*

Answer: In SSR, the server generates the full HTML for a page and sends it to the browser, which is fast for the 'first paint'. However, this HTML is just static; it doesn't have any of the JavaScript interactivity (like 'click' listeners). ****Hydration**** is the client-side JavaScript process where React 'wakes up', takes over the static HTML, builds its Virtual DOM, and attaches all the necessary event listeners. It's the process of 'hydrating' the 'dry' HTML with interactivity.

Q52. What is 'tree shaking', and how does this process help a module bundler like Webpack reduce the final bundle size?

[HARD] | *Category: Frontend Developer*

Answer: 'Tree shaking' is a 'dead code elimination' process. Modern JavaScript uses ES Modules (`import`/`export`). A bundler like Webpack can statically analyze these `import` statements. If you `import` a large library but only use *one* function from it, 'tree shaking' will 'shake' the code to see what 'leaves' (functions) are *not* being used. It will then *exclude* all that unused, 'dead' code from the final production bundle, resulting in a much smaller file size.

Q53. What is 'TypeScript', and what are the main advantages of using a static type system in a large frontend application?

[HARD] | *Category: Frontend Developer*

Answer: TypeScript is a 'superset' of JavaScript that adds 'static types'. All TypeScript code is compiled down to plain JavaScript. **Main Advantages:**

- 1. Early Bug Detection:** It catches 'type' errors (e.g., 'passing a string to a function that expects a number') at 'compile time', before the code ever runs in a browser.
- 2. Better Tooling:** It enables superior 'IntelliSense', autocompletion, and refactoring in code editors.
- 3. Code Readability:** It acts as documentation, making it clear what kind of data a function expects. This is invaluable for large teams and complex codebases.

Q54. How does the 'useEffect' hook in React work? What is the purpose of its 'dependency array', and what happens if you omit it?

[HARD] | *Category: Frontend Developer*

Answer: The 'useEffect' hook runs a 'side effect' function 'after' the component has rendered. The 'dependency array' (the second argument) controls 'when' the effect function re-runs.

- 1. Array with variables [a, b]:** The effect will 'only' re-run if 'a' or 'b' has changed since the last render.
- 2. Empty array []:** The effect will run 'only once', after the initial render (mimicking 'componentDidMount').
- 3. No array (omitted):** The effect will run 'after every single render', which is usually undesirable and can cause infinite loops.

Q55. What is 'accessibility' (a11y), and what are the 'WAI-ARIA' roles and attributes used for? Provide an example.

[HARD] | *Category: Frontend Developer*

Answer: 'Accessibility' (a11y) is the practice of designing and building web applications so that they are usable by everyone, including people with disabilities. **WAI-ARIA** (Accessible Rich Internet Applications) is a specification that helps make complex, dynamic UIs (like tabs or modals) accessible. It uses HTML attributes to add context. For example, if you build a 'tab' component out of `<div>` tags, you must add `role="tablist"` to the container and `role="tab"` to the buttons to tell a screen reader what the component *is*.

Q56. What is 'Cross-Site Scripting' (XSS), and what are the primary measures a frontend developer must take to prevent it?

[HARD] | *Category: Frontend Developer*

Answer: XSS is a security attack where a malicious user 'injects' a client-side script (like JavaScript) into a webpage that is then viewed by other users. This script can steal session tokens, cookies, or perform actions on behalf of the user. **Prevention:** The primary defense is to **never trust user input**. You must *sanitize* or *escape* all user-generated content *before* rendering it into the HTML. Modern frameworks like React do this by default, but you must be careful when using properties like `dangerouslySetInnerHTML`.

Q57. What are 'Service Workers', and what capabilities do they provide for building Progressive Web Apps (PWAs), such as offline support?

[HARD] | *Category: Frontend Developer*

Answer: A 'Service Worker' is a type of 'Web Worker'. It's a JavaScript file that runs in the background, separate from the main browser thread, and acts as a 'proxy' between the browser and the network. This gives it powerful capabilities for **Progressive Web Apps (PWAs)**. Its main use is to intercept network requests (`fetch` events) and serve cached assets from the 'Cache API', allowing an application to work *offline*. It can also manage push notifications.

Q58. Explain the 'critical rendering path' and what steps (like minification, compression, and deferring scripts) you would take to optimize it.

[HARD] | *Category: Frontend Developer*

Answer: The 'critical rendering path' is the sequence of steps a browser must take to render the *initial* view of a webpage. This involves fetching the HTML, parsing it to build the DOM, fetching and parsing the CSS to build the CSSOM, combining them to create the 'Render Tree', and then performing 'Layout' and 'Paint'. **Optimization:** To speed this up, you must **minimize** the size and number of *critical resources*. This includes **minifying** CSS/JS, **compressing** assets (Gzip), and **deferring** or **asyncing** non-critical JavaScript so it doesn't block the initial render.

Q59. What is 'Cross-Site Request Forgery' (CSRF), and how can 'anti-CSRF tokens' be used to mitigate this type of attack?

[HARD] | *Category: Frontend Developer*

Answer: CSRF is an attack that tricks an *authenticated* user's browser into sending a request to a web application that they are logged into, without their consent (e.g., a 'delete account' request hidden in an image tag on a malicious site). **Anti-CSRF Tokens** prevent this. The server generates a unique, secret token and embeds it in the frontend. For any state-changing request (like a `POST`), the frontend must include this token. The server then validates the token, proving the request came from the *actual* application, not a malicious third-party site.

Q60. What are the differences between 'server-side rendering' (SSR) and 'static site generation' (SSG)? What are the pros and cons of each?

[HARD] | *Category: Frontend Developer*

Answer: Both render React/Vue pages to static HTML. **SSR** (e.g., Next.js default): **Pro:** Renders the HTML *on the server* for *every single request*. **Con:** Slower 'Time to First Byte' (TTFB) because it must run code on the server. **SSG** (e.g., Gatsby, Next.js): **Pro:** Renders *all* pages to HTML *at build time*. **Con:** Extremely fast (served from a CDN), secure. **Con:** Not suitable for highly dynamic data (like a user dashboard); you must 're-build' the entire site to update content.